(Chapter 7. Transactions)

A transaction is a way for an application to group several reads and writes together into a logical unit.

Conceptually, all reads and writes in a transaction are executed as one operation, its either the entire transaction Succeeds (commit) or it fails (Rollback, abort).

Purpose of transactions is to simply the programming model for applications accessing a database.

By using transactions, the application has safety guarantees.

With the hype of distributed databases, there emerged a popular belief that any large-scale system would have to abandon transactions in order to maintain good performance and high Availability.

Transactions have their trade-offs.

[The meaning of ACID] → [Refer to video of Hussain Nasser]

Systems that do not meet the ACID criteria are sometimes called BASE.

BASE → [Basically Available, Soft State & Eventual Consistency]

Atomicity → If the writes are grouped together into an atomic transaction, and the transaction cannot be completed (committed) due to fault, then transaction is rolled back, transaction is aborted / rolled back.

Consistency →

Isolation → The Classic Database textsbooks formalize isolation as Serializeability. They say that transactions are not happening concurrently and they are happening one by one. ~~The database ensures~~ → As if they ran serially.

But in reality transactions happen concurrently but they maintain the data in such a fashion ~~that~~ it seems it was serially.

But Serialisation cause database's performance to go down.

Oracle uses "Snapshot Isolation" instead of serialisation.

We will see later in this chapter.

Durability → Enhancing durability by replication does not guaranteee Durability, its just a risk reduction.

Single-Object Multi-Object Operations.

Single Object → Updating 1 key in 1 transaction.

Multi-Object → Updating multi-object in transaction, might be related or unrelated.

Multi-Object transactions require some way of determining which read and write operations belong to the same transaction. In RDMS, it is typically done based on the client's TCP connection to the database server, on any particular connection, everything between a BEGIN TRANSACTION and a COMMIT statement is considered to be part of the same transaction.

(☆ ⊙ ☆) But imagine, if changes are committed but just before sending ack to client the TCP connection breaks. To solve this issue a transaction manager can group operations by a unique transaction identifier that is not bound to a particular TCP connection.

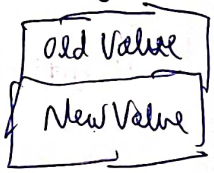
"The-End-to-End-Argument for Databases" ⇒ [Chapter 12]

(Single-Object writes)

Assume you are writing a 20KB Json to a database

3 scenarios

* Network interruption after first 10KB, does the DB store that unparseable 10KB fragment of JSON.
* If the power fails while DB is in middle of overwriting the previous value, do we have spliced value



* Another client reads that document while the write is in progress, will it see a partially updated value?

(***)

Atomicity can be implemented using a log for crash recovery, when server comes back, check the log and Rollback & retry if required.

Isolation can be implemented by using Lock on each object.

Some databases even provide complex atomic operations such as [Atomic Numbers]/[Compare & swap] → Watch Java Playlist of Concepts & Coding.

[Need for multi-object transactions]

There are many cases where mostly single-object operations are done only.

* Foreign key reference to another table, So if one key is updated in one table, maybe another key in another table needs editing.
* In a document, multiple keys in the document, we might need to update
* In DB with multiple secondary indexes, so everytime you update the DB, these indexes need to be updating.

In particular, datastores with leaderless-replication, work much more on a "best effort" basis. If it runs into error the application won't rollback, so its the applications responsibility. Like C* if it is not available to write, on all w nodes on quorum, then it writes to some nodes and does not rollback.

1) Hinted Handoff → drop a hint on online nodes to write on nodes when they come back.
2) Write-Ahead log (WAL) (first write the log then do operation)
3) Read Repair: During Read, if any replica is out of sync, resync it. → LWW (Last Write Wins)

Although retrying is an option, but some cases are there.

* Imagine the network interrupt even though the change got committed, but before the client got Ack. Then a client might retry and double operation will be performed.

* If error is due to overload, then if we retry on the node will put more load on overloaded server

* Its only worth retrying after transient errors [deadlock, tempory network interruptions, failover].

  After permanent Error retry is pointless.

* If transaction has side effects like sending mail, then Retrying on transaction will send mail again.

  this can be prevented by (2PC) → 2 Phase Commit

Weak Isolation Levels

If 2 transactions modify different data, no issues,

If they modify same data then comes trouble. Concurrency bugs are hard to reproduce and hard to find in testing.

Serializable Isolation has performance cost, and it is common for DBs to use weaker Isolation Levels

[Refer Hussain Nasser for this]

[Read Committed]

With this isolation level, It makes a guarantee that a transaction only reads Commited data.

In Read Committed dirty Reads can be prevented.

But can dirty writes be prevented,?

Dirty Writes → (Since transaction is not committed Hence we can prevent It.)

T&1	T&2
SET BALANCE = 100 WHERE ID = 2	
SELECT BALANCE WHERE ID = 2. = 100	
SELECT RATEOFINTEREST WHERE ID = 2 = 5	
SELECT BALANCE * Rate of Interest, Where ID = 2	SET BALANCE = 200 WHERE ID = 2.

If the transaction Happens here, then we have a problem.

In the screen we see

BALANCE = 100, Rate of Interest = 5,

But (BALANCE * Rate of Interest = 100)

Read Committed saves us from dirty Reads and dirty writes, Still there are lot of cases it does not cover.

[Implementing Read Committed]

Most Commonly, databases prevent dirty writes by using row-level locks, when a transaction wants to modify a particular object (row or document), it must first acquire lock on that object, It holds the lock until the transaction is committed or aborted.

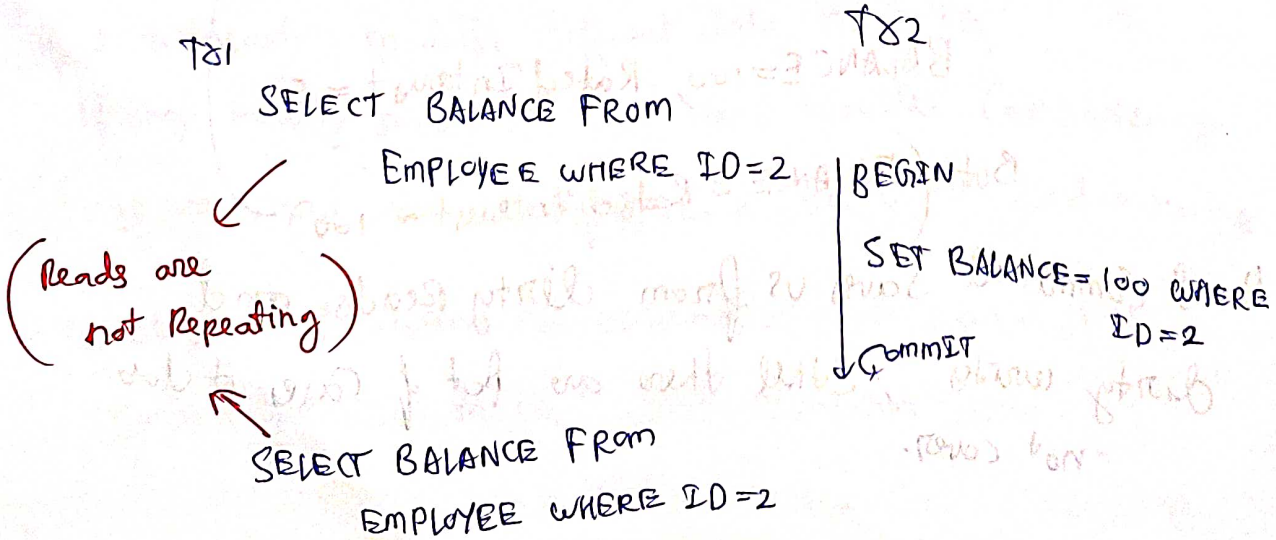
This locking is done automatically by databases in read committed mode [Stronger Isolation].

(** 9)

However the approach of requiring read locks does not work well As a ~~read~~ write query running for too long will acquire the lock for as long as required and harm performance.

Snapshot Isolation and Repeatable Read.

Read Committed does not solve the problem of Non-Repeatable Reads.



[But if ~~ALICE~~ ~~keeps~~ user keeps making the ~~#~~ view BALANCE transaction, again a few times, a he will see consistent result of BALANCE = 100.]

But this Non-Repeatable Read Problem cannot be afforded in many scenarios.

(Example)

1) While taking a backup.

Backup Read BALANCE = 50,

But due to Non-Repeatable Read version got changed while taking Backup of a DB which takes hours to complete. If during that time the DB ~~keeps~~ keeps accepting writes. then BACKUP stores in consistent data,

2) Running Analytics → Not if write transactions can mess with DB making the Analytics report inconsistent

Snapshot Isolation → One ~~way~~ way to prevent Non-Repeatable Reads is that before each transaction, take a consistent snapshot and make changes on the basis of that.

Implementing Snapshot Isolation.

* Snapshot Isolation typically works on the principle of that

1) reads never block writers → { Because they took a snapshot and are using that to read }

2) writers never block readers → { Because writes work on the snapshot they took before any transaction, and same with Readers }.

(***)

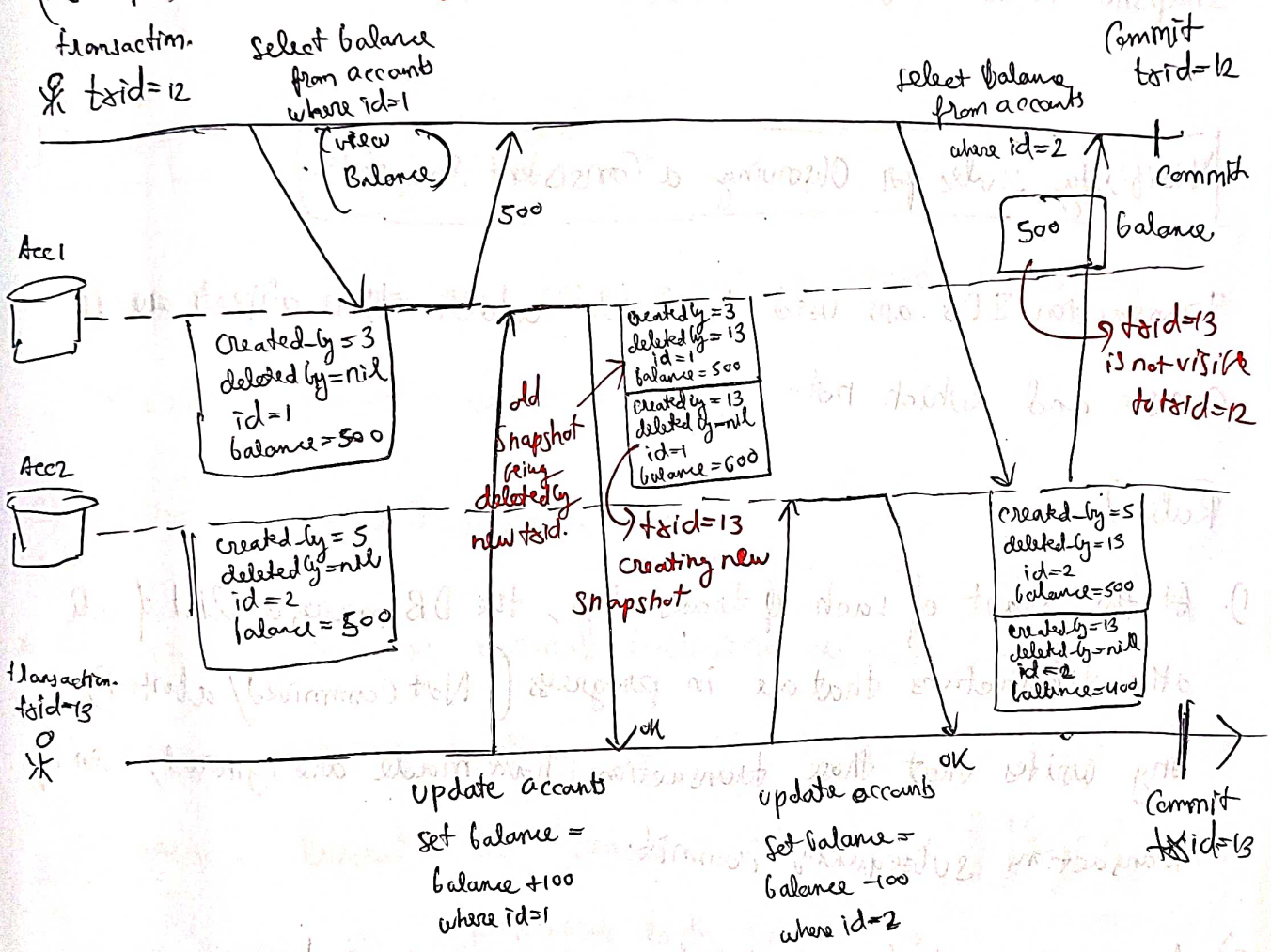
But they use write locks to prevent dirty writes, which means a write transaction blocks the row for another write transaction to complete.

(***)

Since the database has to maintain different committed versions of an object, because various in-progress transaction may need to see the state of database at different points in time. Because it maintains multiple versions of an object side by side, this is called **Multi Version Concurrency Control (MVCC)**.

For implementing transaction, whenever each transaction is starting we assign it with a unique, always increasing tid. Whenever a transaction writes anything to the table, it is tagged with tid of the writer.

(Example) → We are looking at an example of 2 accounts have 500 each. watch how transaction visibility is controlled.



(2020)

Each row in a table has created_by-field, containing ID of the transaction. Moreover, each row has a deleted_by field which is initially empty. It stores the id of transaction which deleted the transaction. the snapshot, (row) is marked for deletion, but it is not actually deleted_by_field. At some later time, when that row is not getting used, the Garbage collector clears it.

A update is internally translated in a delete and a create.

As we saw txid=12, when it tried to see balance of Account id=2, it could not see 400, because txid is bigger than it, and by Snapshot isolation we have made it not appear to it

Visibility rules for Observing a Consistent Snapshot:

Transaction IDs are used to decide which objects it can see and which not

Rules:)

1). At the start of each transaction, the DB makes a list of all other transactions that are in progress (Not commited/aborted). Any writes that those transactions have made are ignored, even if transactions subsequently commit

2). Any writes made by aborted transactions are ignored.

3). Any writes made by transactions with a later transaction ID are ignored, regardless if they have been committed.

4). All other writes are visible to application queries.

A long running transaction may continue using a snapshot for a long time, continue to read values which have been long overwritten or deleted.

Thus database can provide a consistent snapshot while incurring small overhead. → [Hence session expires so fast].

Indexes and Snapshot Isolation.

Since these databases are maintaining version, Hence they are called multi-version Database.

Q) Now how to apply Indexes on these?

* 1st Option is to have normal indexing as usual, and have all versions as they are. The index can have references. Based on the transaction id, we can filter out versions which are not visible to the transactions.

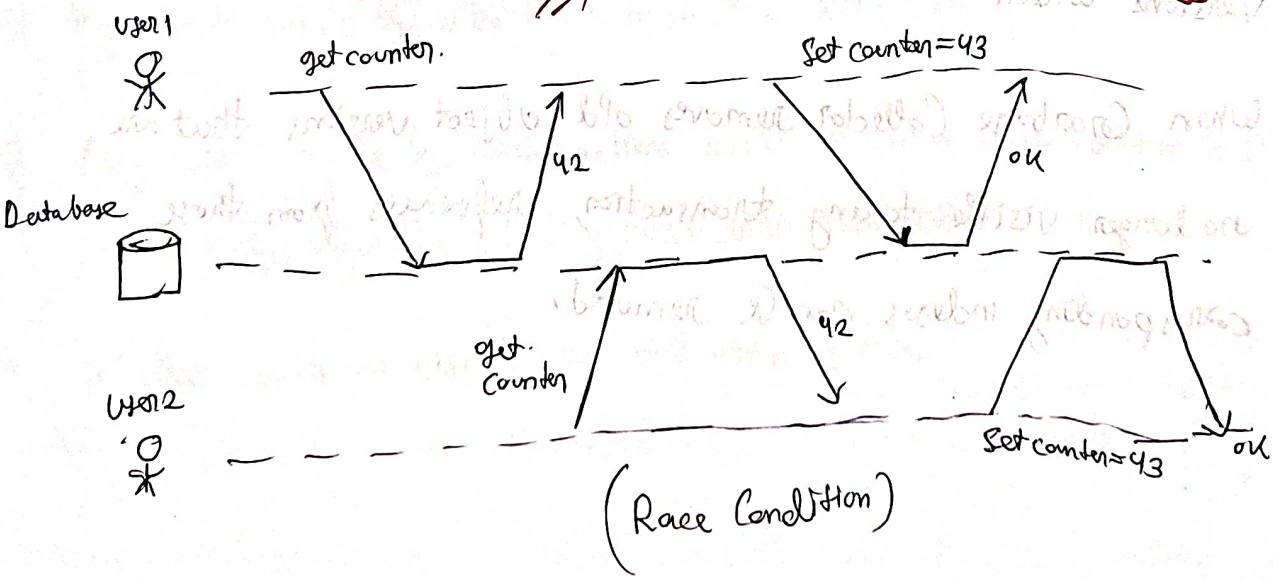
When Garbage Collector removes old object versions that are no longer visible to any transaction, references from those corresponding indexes can be removed.

* Another variant of Databases, they also use B-trees.
using [append-only / copy on write]
this doesn't overwrite pages of the tree where they are updated. Instead the create a modified copy.
Parent Pages, upto the root are copied and updated to new versions. Any pages not affected, don not need to be copied.

* With append-only Btree, for each new version, basically for every transaction a new B-tree is getting created, So we don't need to filter out any transaction and we could use an existing B-tree, which is just the previous version of the incoming transaction. → [But it requires lot of Garbage Collection and Compaction]

Preventing Lost Updates.

(lost update) [Due to concurrent increments]



(Race Condition)

(Read - modify - write) cycle.

* Incrementing a counter or updating an account balance.
* Making a local change to a complex value, eg. adding an element to a JSON Document
* Two users concurrently editing a Wiki Page

(Some Ways)

[Atomic Write Operations]

These are usually implemented by taking an exclusive lock on the object when it is read so that no other transaction can read it until the update has been applied.

Simple way to do it, is by using Single thread.

[Explicit Locking] → If DB does not allow locking, we have to explicitly from application side do locking.

[Automatically detecting Lost Updates] → Leave the responsibility with the transaction manager to detect lost updates and let everything happen in parallel.

[Compare-and-Set] → We read this ~~case~~ in [Java Concept and Coding] As [Compare & Swap].

1. Read Value
2. Modify it
3. During write check if the read value matches the current value.

→ (Compare & Swap)

Conflict Resolution & Replication

All the ways are discussed in the last page,
Atomic operations, Compare & swap. Does not work well
in replicated / Leaderless replication. → Restricted to single Replica.

In Replication, we let all concurrent writes to happen and have different versions and merge them as we saw in Chapter: 5.

LWW → (Last write wins) is prone to lost Updates.

Write Skew & Phantoms

Other than lost updates and dirty writes, there are other scenarios which might happen concurrently.

Write Skew (Hospital On-Call Example)

name	on_call
Alice	true
Bob	true
Carol	false

Alice:

```
begin transaction.
currently_on_call = (
  select count(*) from doctors
  where on_call = true and
  shift_id = 1234
if (currently_on_call >= 2) {
  update doctors set on_call = false
  where name = 'Alice' and shift_id = 1234
}
Commit transaction.
```

Bob:

```
Begin transaction
currently_on_call = (
  select count(*) from doctors
  where on_call = true and
  shift = 1234)
if (currently_on_call >= 2){
  update doctors set
  on_call = false, where
  name = "Bob" and shift = 1234.
Commit transaction.
```

{ Both set oncall to false, Now there is no docto on-call }

(Characterizing write skew)

Anomaly we saw in the example is called write skew.
It is neither a dirty write or lost update because the two transactions are updating two different objects [Alice & Bob's record].

Race Condition: If two transactions had run one after another, then the issue might not have happened.

In write skew, two transactions read the same set of objects and then update some of those objects. [Different transactions updating different objects].

[Difference between write skew & Dirty write]

In dirty write different transactions update the same object.

* Atomic Operations don't help when multiple objects are involved

* If you can't use serializable Isolation Level, 2nd best option is to lock the rows that the transaction depends on.

Meeting Room Booking System → ~~we face~~ We face same issue of concurrent booking of same Room.

Claiming Username → two users getting the same username.

Preventing Double Spending → two users spending 200 Rs even if ~~they~~ balance was there

Multiplayer game:

We can use a lock to prevent lost update [that is making sure that two players cannot move the same figure at the same time]

However, the lock does not prevent players from moving two figures into same position on the board, potentially making some action which violates the rules.

Phantoms causing write Skew

In write Skew we observed the following pattern.

* select to get row on which operation will be done
  In the on call exam, we selected rows where on-call = true.
* Check if Condition is satisfied or not.
* if Yes then Commit, else abort.

What if? We modify the order of execution

1) Do the operation / transaction
2) Check Condition is ok or not
3) Abort or Commit based on Condition.

the concept of locks might come into play here.,
we lock the rows which are going to be used for transaction.

But what about the example of "Claiming a Username", the condition, checks the absence of a row, So you cannot attach lock to anything.

*** this effect where a write in one transaction changes the result of a search query in another transaction, (like doctor on-call example) is called a phantom.

Snapshot Isolation avoids phantoms in read-only scenarios, but in read-write transactions (like doctor on-call example) causes trouble.

Materializing Conflicts

For the booking a meeting room example, to resolve it lets say we add another table of time slots and room separately. Each row in this table corresponds to a particular room for a particular time. we can create schedule for upcoming 6 months. Note this additional table is only used for acquire and releasing a lock, so that something concrete is there. this approach is called materializing conflicts.

Serializeability

Serializable isolation is usually regarded as the strongest isolation level. It guarantees that even though transactions may execute in parallel, the end result is same as if they executed one at a time (serially without Concurrency).

If Serializeable Isolation is so good, why is it not used widely?

To answer this we will have to look at the 3 ways of implementing it
(3 techniques.

1) Literally executing in serial order

2) Two Phase Locking (2PL)

3) Optimistic Concurrency Control such as
SSI → (Serializable Snapshot Isolation)

Actual Serial Execution

Here we will discuss with respect to Single Node

In this technique as the name says, we actually execute transaction 1 by 1 serially. So its Single threaded Basically

What made database Engines switch from multi-threaded to single-threaded transactions?

* RAM became cheap. Now In memory transaction could be done.
* DB designers realised OLTP has short lived transactions, So Single thread is fine. For long running analytic queries which are typically read only. (Snapshot Isolation is fine)

Redis uses this approach of executing transactions serially. A system designed for single threaded execution can sometimes perform better than a system that supports concurrency, because it can avoid the coordination of locking. However, throughput is limited to that of Single CPU Core

Encapsulating transactions in stored procedures

Let's take flight booking system as an example..

During earlier times to avoid any race condition, it was decided that whole customer journey of booking a transaction will be single transaction.

- → Viewing flights
- → Selecting Flight
- → Book tickets
- → Paying upfront

} → Single transaction.

Unfortunately we cannot rely on humans. ~~They~~ to complete transactions at one go. So we avoid Interactive transaction with user, as they can mess with the system.

DB avoids interactively waiting for user for transaction. New HTTP request for each transaction can be used.

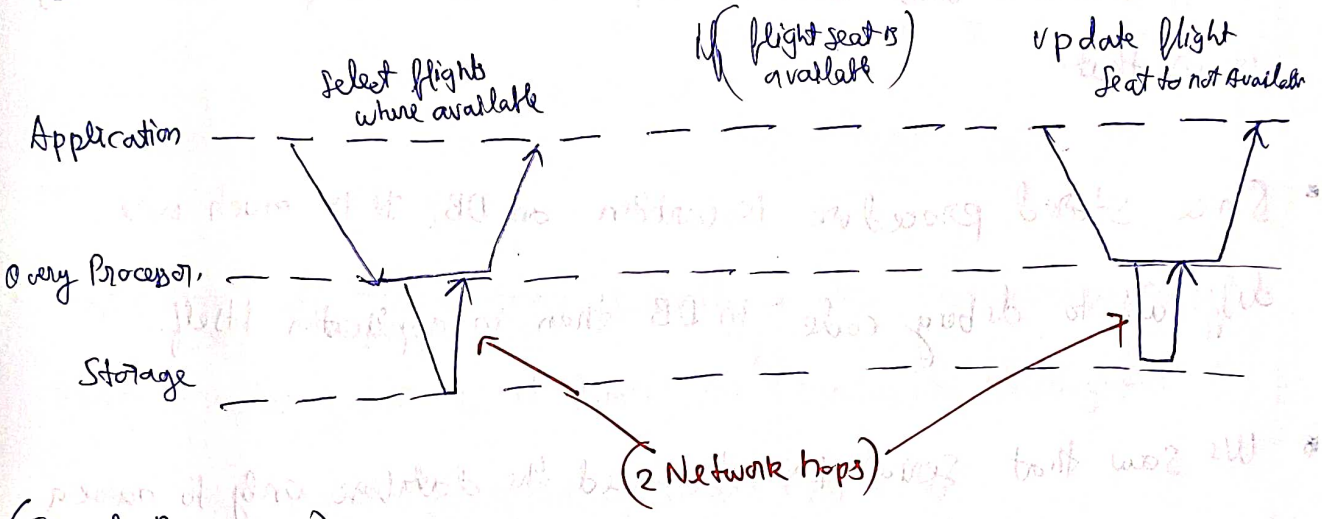
But, even if customer does not interactively communicate with DB, but ~~DB communicates~~ User communicates with application and application in turn with DB.

In interactive style of transaction, a lot of time is spent in network communication between Application and the database, so if we disallow concurrency in the database and only process one transaction at a time that is very bad latency.

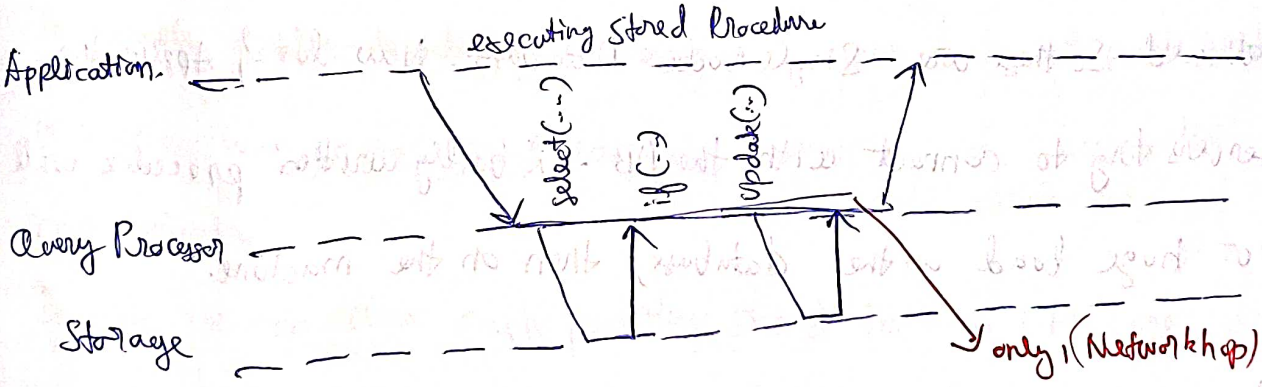
For this reason, single-threaded serial transactions do not allow interactive multi-statement processing.

As we saw interactive multi-statement on single threaded due to huge (network / I/O) calls is a dreadful performance.

(Interactive transaction:)



(Stored Procedure)



[Stored Procedure are directly run on the DB and not on the Application]

Pros and Cons of Stored Procedures

Cons:

* Each DB has its programming language, And stored procedures need to be written in the language of the DB, which have become pretty archaic and developers are not comfortable writting it in that.

* Since stored procedure is written on DB, it is much more difficult to debug code in DB than in application itself

* We saw that serial Execution need the database only to have a single query that also stored procedure, so multiple nodes are not allowed. So these are Single Node Databases. Now lot of Application servers try to connect with the DB. A badly written procedure will put huge load on the database, then on the machine

Pros:

* Modern Implementations of stored procedures have abandoned PL/SQL and use existing Programming Languages → Redis uses Lua

* With stored Procedure and in-memory data, we don't need to wait for Network or I/O calls.

* VoltDB also uses stored procedures for replication: instead of copying a transaction's write from one node to another, it executes the same stored procedure on each replica.

Stored replicas should be deterministic, i.e. when run on different nodes, they must produce same result like timestamp or Rand() should return same values in all replicas

Partitioning

Executing all transactions serially makes concurrency control much simpler, but it limits the transaction throughput which is decided by speed of single CPU core

To enable multi-threading we could have our data set partitioned in such a way so that any transaction doesn't need cross partition communication.

In this we can give each partition/shard its own CPU core, where the single thread will do its operation. → ScyllaDB does it this way.

If transaction is required to operate across partitions, then locks will be required across partitions which is added Loaded

(Summary of Serial Execution)

* Every transaction must be small and fast, as problem of Snapshot Isolation, write skew.
* In-memory processing only
* Partition data in such a way that each partition can have data on each CPU core → to use multi threading
* Cross-partition transactions are possible, but hard limit on how much can be done, as cross partition locks are required.

Two-Phase Locking (2PL) → {this is different from 2 Phase Commit}

Two phase locking is ~~different~~ quite similar acquiring write lock

1. If transaction A has to read an object and transaction B wants to write to that object, B must wait to acquire lock.
2. If transaction A is writting something to an Object and transaction B wants to read that object, B must wait till A releases the lock

2PL (writers block both reads and writes.)
(readers blocks writes but never readers)

Implementing (2PL) → It is same as Shared and Exclusive lock we've read in Java playlist Concept & Coding.

If Object acquires Shared Lock, then another shared lock can acquire the lock. but exclusive lock cannot get it

If Object acquires exclusive lock, then not even shared lock can acquire it, neither exclusive lock can acquire it

Shared Lock → Read Lock

Exclusive Lock → Write Lock.

In (2 phase Lock) we might run into dead lock situation.

2PL does automatic deadlock detection, and root causing transaction which is causing the dead lock is aborted and retried.

Performance of 2Phase Locking

- Performance is Bad.
- Reduced Concurrency because of Overhead of Acquiring lock.
- 2 Phase Lock might make the DB wait and not able to keep transactions short as it might be waiting to get an exclusive lock
- Deadlocks much common in 2PL
- For this reason 2PL have quite unstable latencies.

Predicate locks

Take the meeting room Example. (Its okay to concurrently insert bookings for other rooms, or for the same room at a different time that doesn't affect the proposed booking).

Predicate lock works like shared & Exclusive lock only, but rather than blocking a single Object, it blocks all objects satisfying same condition (predicate).

[SELECT * FROM BOOKINGS WHERE ROOM_ID = 123 AND end_time > time1 AND starttime < time2] ⇒ Shared lock is acquired on this matching objects

(to get exclusive lock it must wait for set of Objects to release shared lock.

Index-Range locks → Using locks on the whole Index

* Predicate locks are slow, do not perform well, Checking for matching locks becomes time consuming. For this reason 2PL uses index-based locking
* Index-Range lock is covering a superset of objects which the predicate lock covered.

Let's say you want to book room 123 between noon to 1p.m. and if index is done by room number then it is much easier to lock that index [lock all bookings] for that transaction.

Index-Range locks lock objects bigger than necessary but provide a very low overhead, so its a good compromise.

Serializeable Snapshot Isolation → [Used in both single-node & distributed]

1. We saw if we want better performance by having weak isolation, we have to suffer concurrency issues.
2. If we have strong Isolation we have performance issue

We have to find a middle-ground between these two approaches

SSI claims to be very promising and it provides full serializeability and only small performance penalty

Pessimistic V/s Optimistic Concurrency Control

Refer to Java Playlist of Concept and Coding

{2PL, Serial Execution → Pessimistic Locks}

Serializeable Snapshot Isolation → Optimistic Concurrency Control

Its like stamped lock, take snapshot before transaction, make required changes,

Before Committing check if current picture matches the snapshot taken at the start. If yes Commit/ else Abort

But it performs badly if there is high contention (too many transactions trying to access the same Object).
This leads to high number of transactions to get aborted,

~~If we~~ Yeah, we can always retry but if we are retrying on an already overloaded machine, things can be tough.

However if contention is not too high we can do it it

(Z 3 8)

On top of Snapshot Isolation, SSI adds an algorithm for detecting conflicts among writes and reads and aborts/rollsback transactions as required.

Decisions made on outdated premise

Basically in the on call example, the decision which is being made, is being made, is on the basis of an outdated snapshot. Number of people currently on-call might have changed.

Few ways in which query result knows snapshot is outdated?

- Detecting Stale MVCC reads
- Detecting Writes Affecting Prior Reads.

Detecting Stale MVCC reads

In Snapshot Isolation each transaction has a particular version (snapshot) of the DB, isolated to the transaction. Implemented by [Multi-Version Concurrency Control (MVCC)]

In the on-call doctor example, If you remember, It was that the 2nd transaction was reading an older Snapshot showing both Alice & Bob oncall, which is outdated.

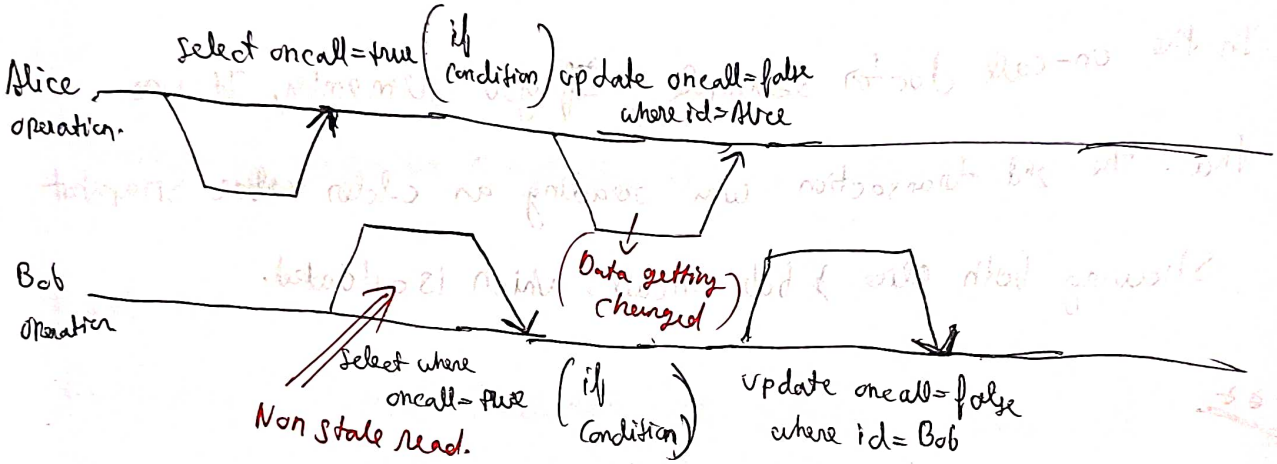
In order to prevent this anomaly, the DB when it goes to commit the change of transaction 2, it checks whether any of the ignored writes have been committed or not. [writes which were ignored so that we get all committed changes Snapshot]. If any ignored transaction has been committed. We have got to abort transaction. Since we know premise is outdated

Q) Why go till the commit stage, why not abort when stale read is detected? → (to avoid unnecessary Aborts)

A) If transaction 2 is read only we need not worry. Cause its not a write-skew. if during stale read, if the earlier transaction which was changing oncall of Alice got aborted, then we would unnecessarily revert oncall of Bob as well.

Detecting writes that affect prior reads

This is the case of not a stale read, but data got modified after a read has been performed. So the read is not stale, but post read the snapshot changes.



Lets say we have indexed the entry by shift id. When a transaction writes to the database, it must look in the indexes for any other transactions that have recently read the affected data. So instead of blocking, here it acts a tripwire. It simply notifies the transactions that data read ~~might not be up~~ has scenario changed now. Alice transaction, Bob transaction both are notified now and decide amongst themselves which to commit which to abort

Performance of Serializable Snapshot Isolation → SSI winning over 2PL and Serial Execution

Compared 2PL, ~~stop~~ SSI does not block any thread. This design makes query latency much more predictable and less variable.

Compared to Serial Execution, SSI is not limited to be single threaded or having [single partition run on each core].